

Tokenization

Tokenization in NLP

- **Tokenization:** splitting a string into a sequence of tokens
- Tokenization is elusively difficult and often requires many specialized rules
- **The simplest approach:** split by white spaces
 - What about punctuation?
 - We could handle punctuation differently, but this still leaves many borderline cases:
 - “amazing!”, “state-of-the-art”, “un-thinkable”, “prize-winning”, “aren’t”, “O’Neill”

Tokenization in NLP

- What do we do when we encounter an unknown word during test time? (i.e., a word that did not appear during training)
 - We could use a special <UNK> token, or use pseudo-words where appropriate
- However,
 - Pseudowords are mostly relevant for certain categories
 - What about other categories?
- Wordform tokens do not tie together different forms of the same stem
 - opens / opening / opened / ...
 - ... אכלתי / אכלנו, אוכל, נאכל, יאכלו ...

Character-level Tokenization

- Split by characters

['S', 'p', 'l', 'i', 't', ' ', 'b', 'y', ' ', 'c', 'h', 'a', 'r', 'a', 'c', 't', 'e', 'r', 's']

- Implications:
 - Dramatic decrease in vocabulary size
 - Much longer input sequences
 - Need to learn from scratch how to combine characters to recover word meaning

Tokens may include multiple words

- Frequent multi-word spans can become single tokens if the tokenizer permits merges across whitespace

Word-Pieces	Words	Multi-Word Expressions
z	_cat	_ever_so_slightly
_pre	_descriptive	_Tony_Stark
ism	_System.out.println	_marketing_campaign
tion	_Pokemon	_Higgs_boson
-ness	_e.g.	_production_capacity
-in-law	😊	_stepping_down
-on-demand	_https://github.com/	_bank_accounts
-of-the-	_LGBT	_National_Public_Radio

Table 3: Examples of items from J1’s vocabulary, including word-pieces, whole words, and multi-word expressions, where we use the more loose sense of a word as a sequences not containing whitespace in the middle of it. Underscore represent a literal space in the token.

Subword Tokenization

- Subword tokenization is a middle ground between word-level and character-level tokenization
- First developed for machine translation -based on byte pair encoding (Gage, 1994)

Neural Machine Translation of Rare Words with Subword Units

Rico Sennrich and Barry Haddow and Alexandra Birch

School of Informatics, University of Edinburgh

{rico.sennrich,a.birch}@ed.ac.uk, bhaddow@inf.ed.ac.uk

The main motivation behind this paper is that the translation of some words is transparent in that they are translatable by a competent translator even if they are novel to him or her, based on a translation of known subword units such as morphemes or phonemes.

Byte Pair Encoding (BPE)

Require: K

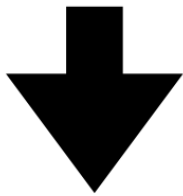
▷ Vocabulary size

- 1: $V \leftarrow$ all characters in the training data
- 2: **while** $|V| \leq K$ **do**
- 3: Pre-tokenize and tokenize the data by repeatedly taking the longest matching prefix
- 4: Count the frequency of all adjacent token pairs in the data
- 5: Let $\langle l, r \rangle$ be the most frequent adjacent token pair
- 6: $V \leftarrow V \cup \{lr\}$ ▷ Add the merged token as a new token
- 7: **end while**
- 8: **return** V

Simple Execution

$K = 10$

Word	Frequency
hug	10
pug	5
pun	12
bun	4
hugs	5



$\mathcal{V} = \{b, g, h, n, p, s, u\}$

$K = 10$
 $\mathcal{V} = \{b, g, h, n, p, s, u\}$

Word	Frequency
h+u+g	10
p+u+g	5
p+u+n	12
b+u+n	4
h+u+g+s	5

Pair	Frequency
u+g	20
p+u	17
u+n	16
h+u	15
g+s	5
⋮	

$$K = 10$$

$$\mathcal{V} = \{b, g, h, n, p, s, u, ug\}$$

Word	Frequency
------	-----------

h+ug	10
------	----

p+ug	5
------	---

p+u+n	12
-------	----

b+u+n	4
-------	---

h+ug+s	5
--------	---

Pair	Frequency
------	-----------

u+n	16
-----	----

h+ug	15
------	----

p+u	12
-----	----

p+ug	5
------	---

ug+s	5
------	---

⋮

$$K = 10$$

$$\mathcal{V} = \{b, g, h, n, p, s, u, ug, un, hug\}$$

Word	Frequency
------	-----------

hug	10
-----	----

p+ug	5
------	---

p+un	12
------	----

b+un	4
------	---

hug+s	5
-------	---

Subword Tokenization

- To avoid unknown words, we must add all possible characters/symbols
- But, there are ~138K unicode symbols
- Instead, use bytes!
 - LLMs often use BPE with some rules to prevent certain types of merges
 - Modest-sized LLMs often have vocabulary sizes of 32-64K
 - With the big proprietary models, it is often ~100k–256k tokens

Subword Tokenization

- Other subword tokenization methods:
 - WordPiece (Schuster et al., 2012): merge to increase likelihood as measured by a language model (vs. frequency as in BPE)
 - SentencePiece (Kudo et al., 2018): can do subword tokenization without pre-tokenization (i.e., using white spaces) - good for languages without such word boundaries – Although pre-tokenization still often helps.

Morphologically-aware Tokenization

- In morphologically-rich languages, especially ones where the morphology is not concatenative, the use of morphological segmentation may be helpful
- For example, the Arabic-centric Fanar model uses a modification to BPE, motivating it with this example:

الرحمن ← ال + رح + من
In Hebrew transliteration: אל-רחמן ← אל + רח + מן

Morphologically-aware Tokenization

Algorithm 1 Morphologically-aware BytePair Encoding (MorphBPE)

- 1: Initialize vocabulary with individual characters
 - 2: Segment the training corpus using morphological segmentation
 - 3: **while** number of merges < desired vocabulary size **do**
 - 4: Compute byte-pair frequencies
 - 5: **Modified Step:** Find the most frequent byte pair that does not cross morpheme boundaries
 - 6: Merge the most frequent byte pair into a new symbol
 - 7: Update the vocabulary with the merged symbol
 - 8: **end while**
-

الرحمن ← ال + رحمن (ال + رحمن)

- This way, cases of letter sequences that are incomplete parts of morphemes will not be counted with their instances as complete morphemes

Morphologically-aware Tokenization

- Similar benefits from preceding the statistical tokenization with morphological segmentation are shown for Hebrew (in this case WordPiece and not BPE)

Prefix	Suffix	Form	English Translation	WordPiece Tokenization	Morphological Segmentation Tokenization	Prefix-Suffix Separation Tokenization
-	-	שחרור	liberation	שחרור	<u>שחרור</u>	<u>ש</u> +חרור
ש	-	ששחרור	that lib.	ששחר ##ור	<u>ש</u> +שחרור	ש+שחרור
ו	ה	ושחרורה	and her lib.	ושחרור ##ה	ו+שחרור+ה	ו+ש+חרור+ה
ו+כ	נו	וכשחרורנו	and as our lib.	וכש ##חרור ##נו	ו+כ+שחרור+נו	ו+כ+ש+חרור+נ+ו
(1)	(2)	(3)	(4)	(5)	(6)	(7)

Conclusion

- Tokenization is an essential preprocessing step for training neural language models
- A variety of methods are employed, from character-level vocabularies to inclusive vocabularies that also include multi-word expressions
- The common practice is using statistical methods, such as BPE (byte pair embeddings)
- Languages with non-concatenative morphology may require a morphological segmentation step
- Tokenization becomes more important when training data is not as abundant, i.e., low-resource languages